

Operaciones de Aprendizaje Automático III

Entornos Reproducibles y Gestión de Modelos

MSc. Fernando Silva



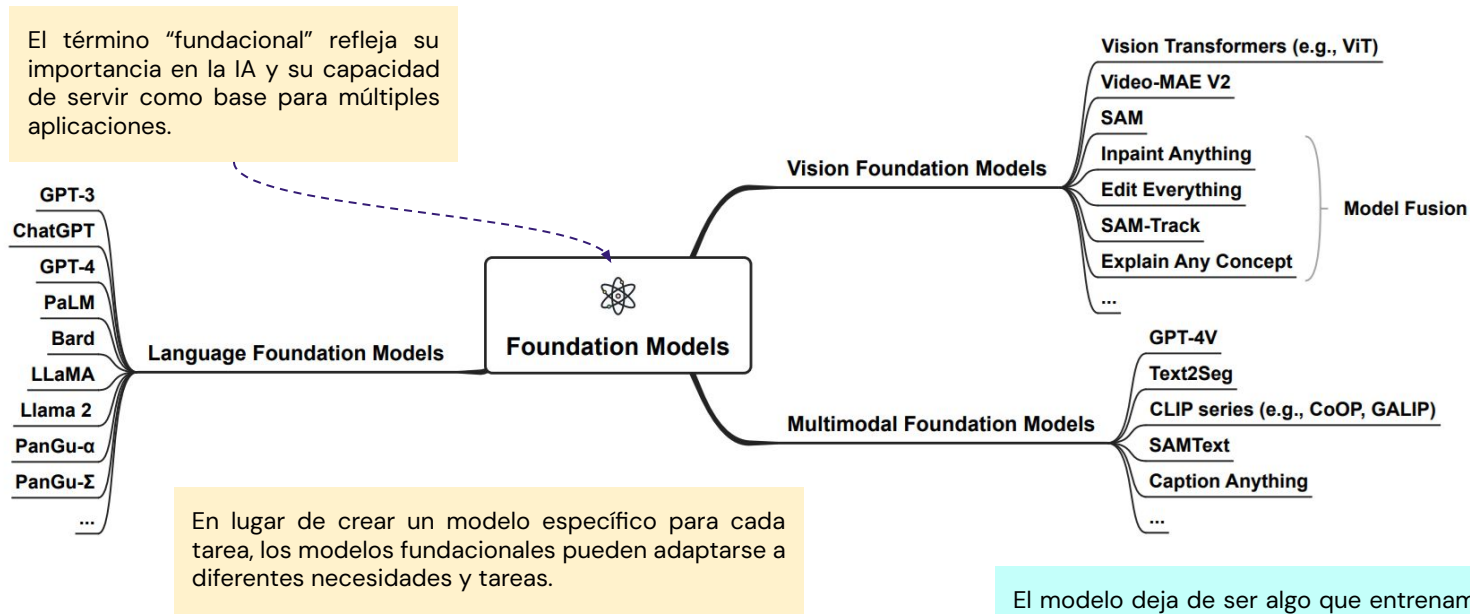
Agenda

- Repaso
- Arquitectura de Transformers
- Anatomía de una Inference Pipeline
- Reproducibilidad
- Gestión de dependencias
- Descarga y manejo de modelos
- Model registries y versionado

Repaso

Foundation Models

Los modelos fundacionales (Foundation Models) surgieron a partir de los grandes modelos de lenguaje (LLM), que a su vez se originaron como simples modelos de lenguaje.



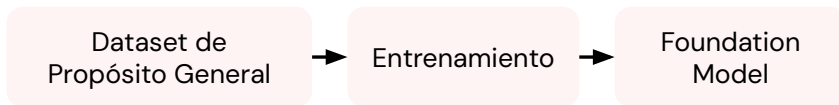
MLOps vs LLMOps

MLOps (Machine Learning Operations) es el conjunto de prácticas, herramientas y procesos para llevar modelos de Machine Learning desde el desarrollo hasta producción y mantenerlos funcionando de forma confiable.

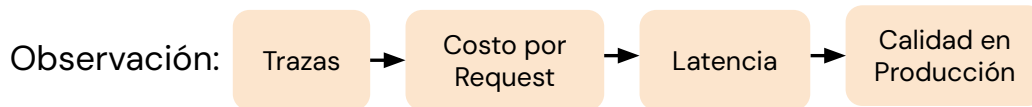
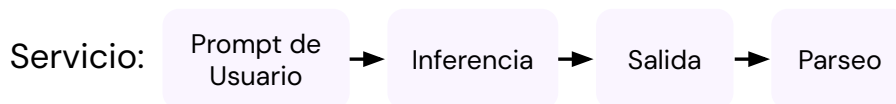
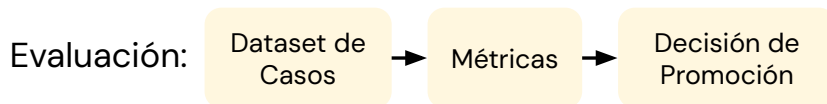
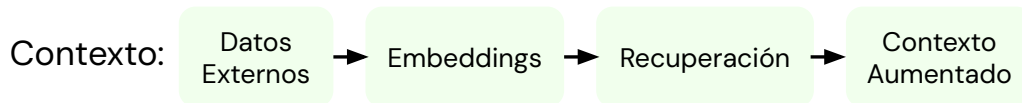
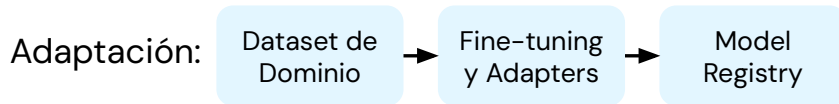
LLMOps (Large Language Model Operations) es la aplicación de principios de MLOps específicamente a LLMs y aplicaciones basadas en LLMs

MLOps vs LLMOps

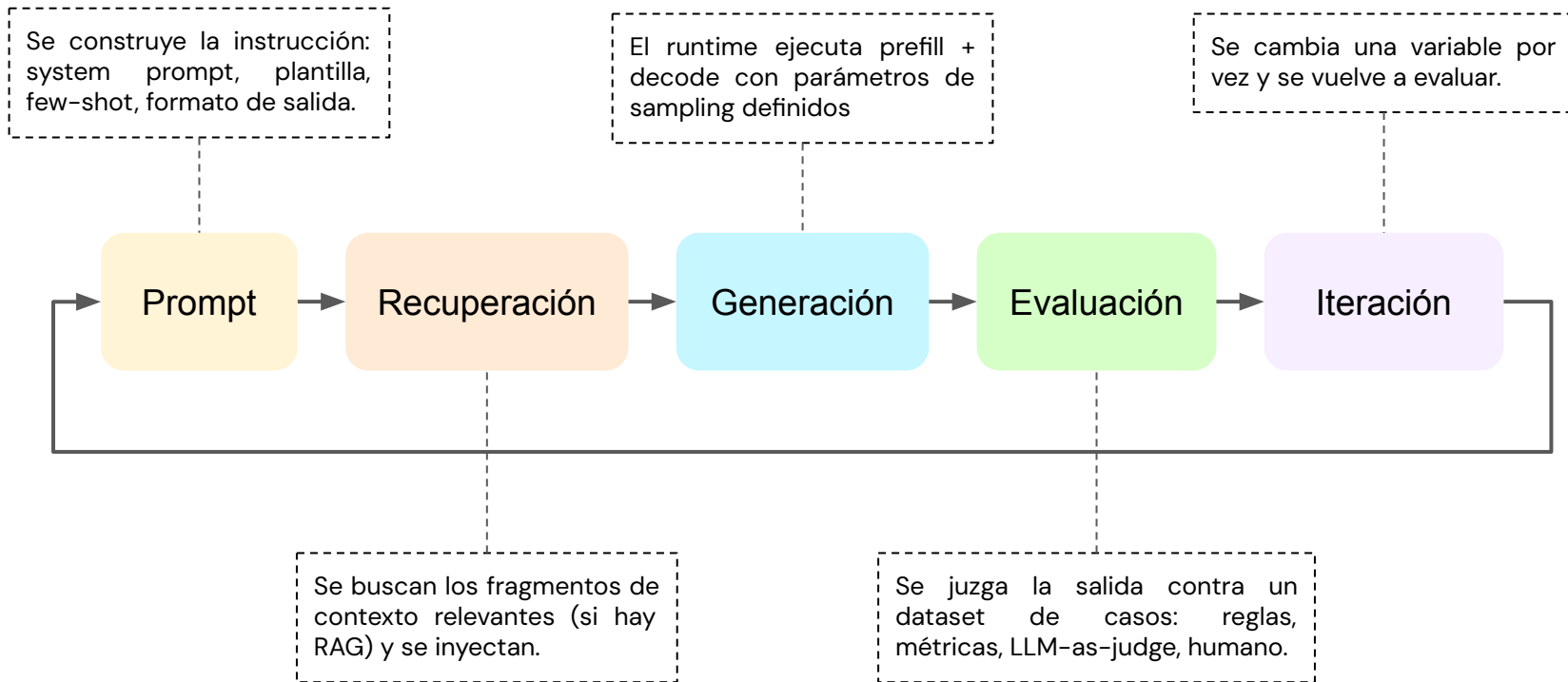
Fuera de nuestro control:



Dentro de nuestro control:



Ciclo de Vida de una Aplicación LLM



Diferencia clave con MLOps: el loop de iteración dura minutos, no días. No hay reentrenamiento en el medio. Eso hace que el cuello de botella se mueva del cómputo a la evaluación.

Prompting, RAG y Fine-Tuning

Fácil y barato



Difícil y costoso

Prompting

RAG

Fine-tuning

Tipos de Open Source

Categoría	Se publica	Se puede auditar	Licencia típica
Open source estricto	Pesos, código y datos	Todo	Apache 2.0, MIT
Open weights	Solo los pesos	El comportamiento, no el origen	Apache 2.0, MIT
Open weights restringido	Pesos con condiciones de uso	El comportamiento	Licencia comunitaria propia
Cerrado	Nada	Nada	Términos de servicio

La gran mayoría de los modelos que se llaman "abiertos" pertenecen a la segunda o tercera categoría.

Ventana de Contexto

La ventana de contexto es la cantidad máxima de tokens que el modelo puede procesar en una sola llamada.

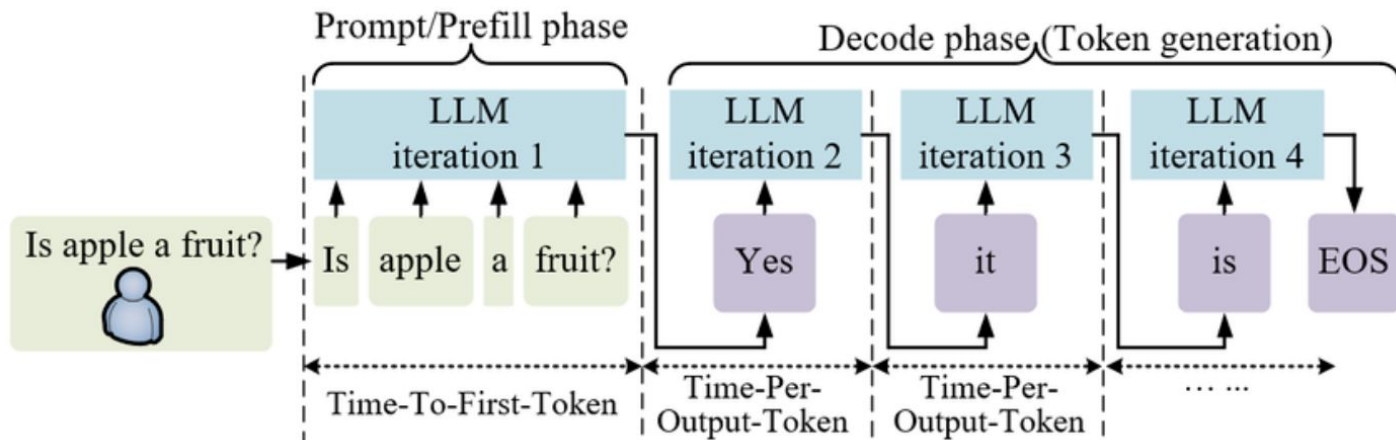
- **Costo cuadrático de atención:** el mecanismo de self-attention compara cada token con todos los demás, por lo que el cómputo (y en la práctica la latencia del prefill) crece con el cuadrado de la longitud del contexto. Duplicar el contexto no duplica el costo, lo multiplica más que proporcionalmente.
- **"Lost in the middle":** modelos con ventanas largas muestran peor recall sobre información ubicada en el medio del contexto que sobre lo que está al principio o al final.

	the	cat	sat	on	the	mat
the	-	low	low	low	-	low
cat	high	-	high	medium	low	medium
sat	low	high	-	high	low	medium
on	low	medium	high	-	low	high
the	-	low	low	low	-	low
mat	low	medium	medium	high	low	-

Inferencia en los LLMs

La inferencia consiste en utilizar un modelo de machine learning previamente entrenado para procesar datos nuevos que no ha visto durante su entrenamiento.

En el caso de los LLM, la inferencia ocurre cuando se recibe una entrada del usuario, como un prompt, y esta se procesa mediante los parámetros aprendidos por el modelo para generar una respuesta.



Inferencia en los LLMs

Las cuatro métricas que vamos a monitorear toda la materia:

- **TTFT (Time To First Token):** Cuánto tarda en aparecer el primer token. Domina la percepción de "responde rápido". Depende del largo del prompt (prefill). Es lo que empeora cuando inflamamos el contexto.
- **TPOT (Time Per Output Token):** Cuánto tarda cada token siguiente. Define la sensación de fluidez del streaming. Depende del tamaño del modelo y del hardware, no del largo del prompt.
- **Latencia total = TTFT + (TPOT × tokens generados)**
- **Throughput:** tokens/segundo agregados en todo el servidor. Es la métrica de **costo**. Cuántos usuarios sostiene una GPU.

Trade-off: optimizar latencia (batches chicos, poca espera) y optimizar throughput (batches grandes, más espera) son objetivos opuestos. Todo runtime de serving es una posición elegida sobre ese eje.

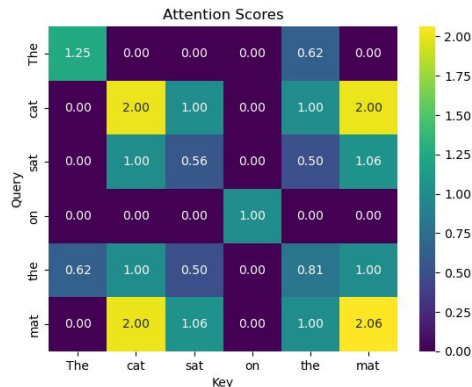
Arquitectura de Transformer

Arquitectura de Transformer (Parte Operativa)

Las cuatro propiedades que explican todo el comportamiento operativo que ya se vio previamente:

1. Self-attention compara cada token con todos los demás.

- El prefill es cuadrático en la longitud del prompt.
- El TTFT crece más que proporcionalmente con el contexto.
- Esto explica lo que vimos en la slide de ventana de contexto: duplicar el contexto no duplica el costo, lo multiplica más que proporcionalmente.

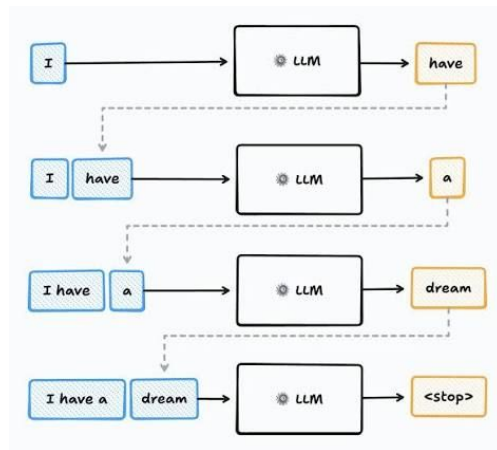


Arquitectura de Transformer (Parte Operativa)

Las cuatro propiedades que explican todo el comportamiento operativo que ya se vio previamente:

2. La generación es autorregresiva: un token por vez, cada uno condicionado al anterior.

- La salida no se puede paralelizar
- Duplicar los tokens de salida duplica la latencia, en cualquier hardware

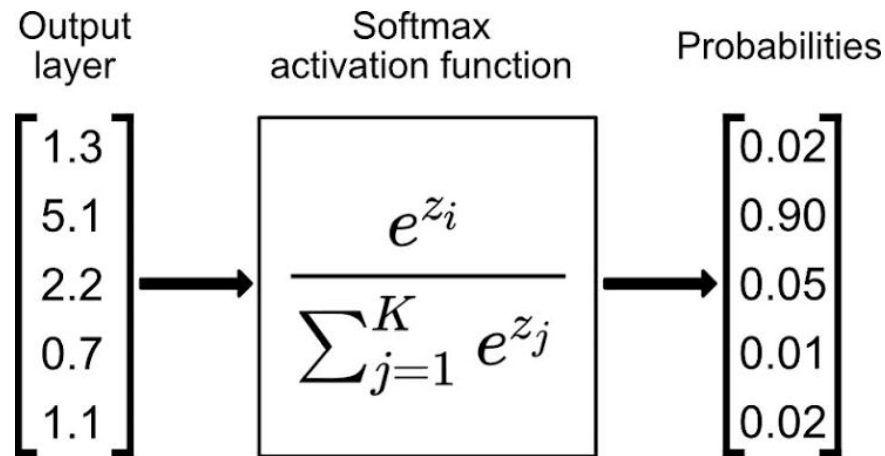


Arquitectura de Transformer (Parte Operativa)

Las cuatro propiedades que explican todo el comportamiento operativo que ya se vio previamente:

3. El modelo produce una distribución de probabilidad sobre el vocabulario, no una respuesta

- La respuesta aparece recién cuando se muestrea de esa distribución.
- El no determinismo es una propiedad del sistema



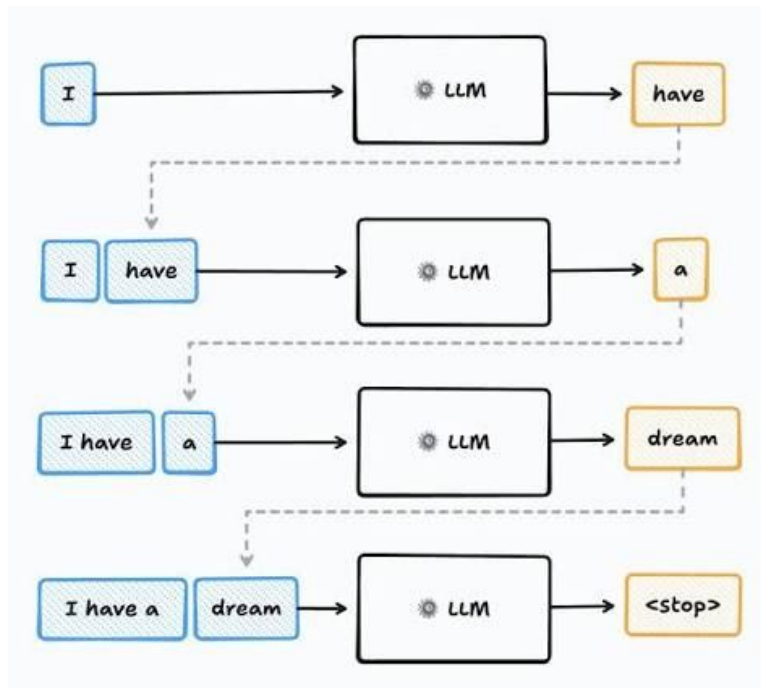
Arquitectura de Transformer (Parte Operativa)

Las cuatro propiedades que explican todo el comportamiento operativo que ya se vio previamente:

4. La generación se sostiene sobre un cache de estado: el KV cache

En cada paso de decode, el modelo necesita atender a todos los tokens anteriores. Recalcular sus proyecciones K y V en cada paso sería cuadrático otra vez. En vez de eso se guardan y se reutilizan.

- Ese cache es el que hace que el decode sea lineal por token y no cuadrático.
- Su tamaño crece con el contexto y con la cantidad de secuencias concurrentes.
- Es memoria de GPU que no es del modelo.



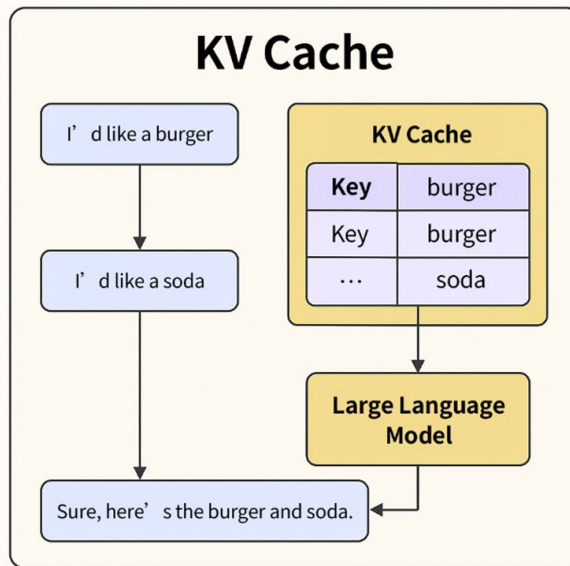
Arquitectura de Transformer (Parte Operativa)

Las cuatro propiedades que explican todo el comportamiento operativo que ya se vio previamente:

4. La generación se sostiene sobre un cache de estado: el KV cache

Consecuencias operativas:

- La memoria de inferencia crece con contexto por batch.
- El límite de concurrencia de un servidor casi nunca son los pesos: es el KV cache.
- Por eso existe vLLM y por eso existe PagedAttention.



Anatomia de Inference Pipeline

Prompt Template vs Chat Template

Prompt Template: una plantilla para construir un prompt de texto.

Chat Template: una plantilla para convertir una conversación con roles (system, user, assistant) al formato que entiende un modelo de chat.

Prompt Template

Es simplemente un texto con variables que podemos rellenar.

Principalmente para reutilizar prompts

```
Input:
Exemplar(s):
Context: [Original Context]
Question: [Question]
Answer: [Answer]
...more exemplars...
Context: [Original Context]
Question: [Question]
=====
Output:
```

Construye el texto

Chat Template

Es una plantilla que organiza los mensajes de una conversación (system, user, assistant) en el formato específico que espera el modelo.

```
<|im_start|>system
You are a helpful assistant.<|im_end|>
<|im_start|>user
Hello!<|im_end|>
<|im_start|>assistant
Hi! How can I help you today?<|im_end|>
<|im_start|>user
What's the weather?<|im_start|>assistant
```

Estructura la conversación para el modelo.

Anatomia de Inference Pipeline

Todo lo que ocurre entre el input del usuario y el output de nuestra aplicación:

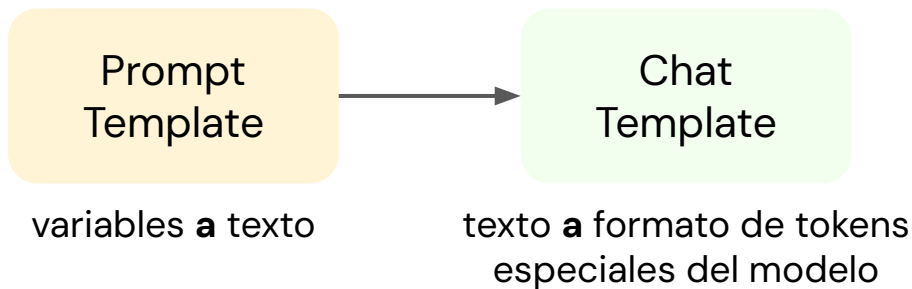


Prompt
Template

variables **a** texto
(system + user + few-shot)

Anatomía de Inference Pipeline

Todo lo que ocurre entre el input del usuario y el output de nuestra aplicación:



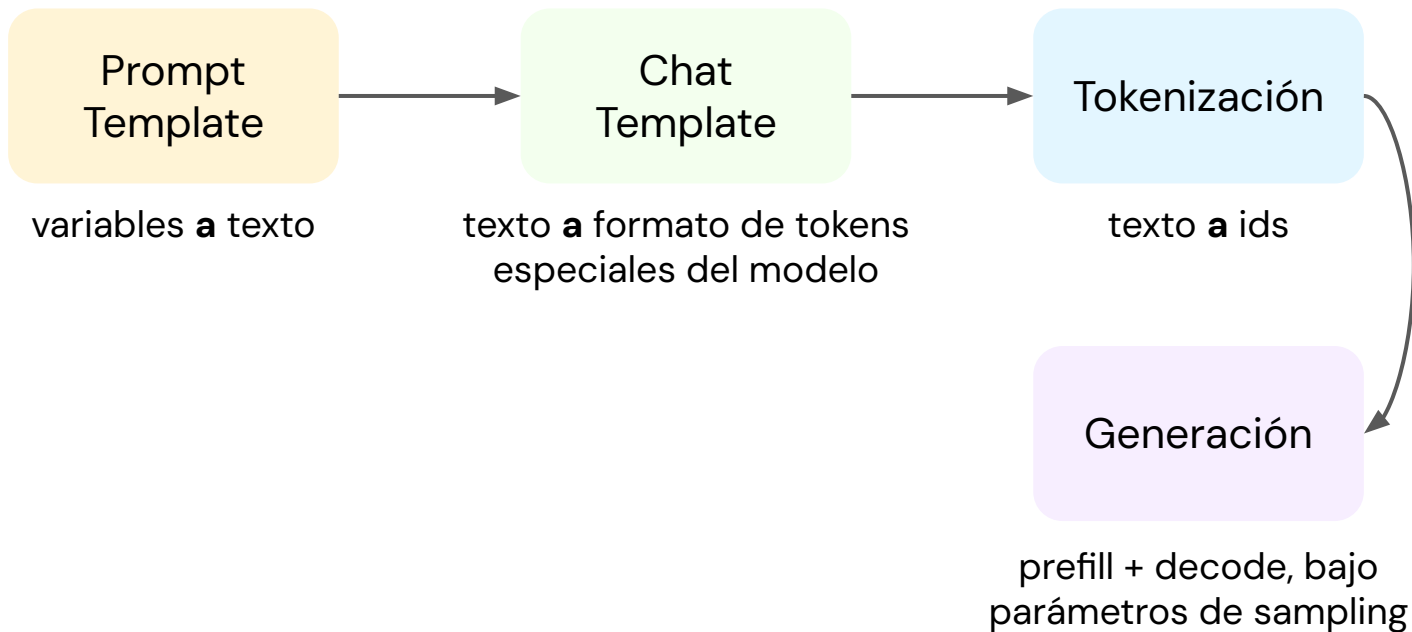
Anatomía de Inference Pipeline

Todo lo que ocurre entre el input del usuario y el output de nuestra aplicación:



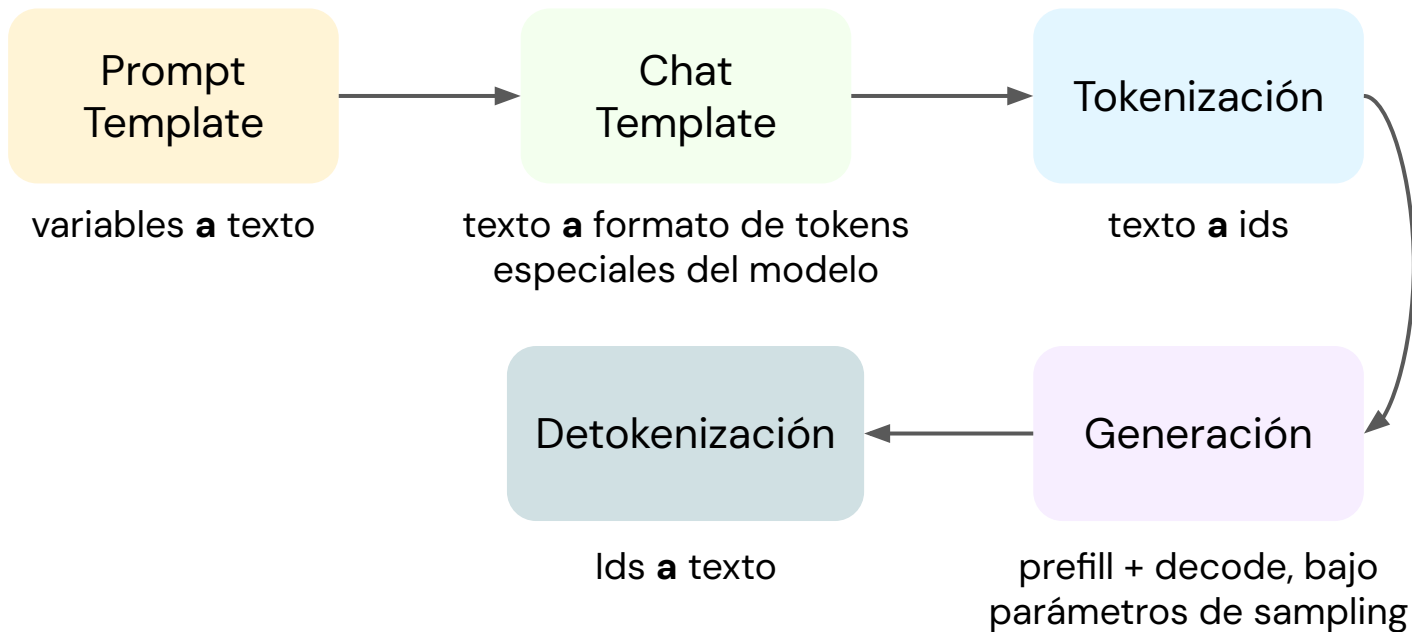
Anatomía de Inference Pipeline

Todo lo que ocurre entre el input del usuario y el output de nuestra aplicación:



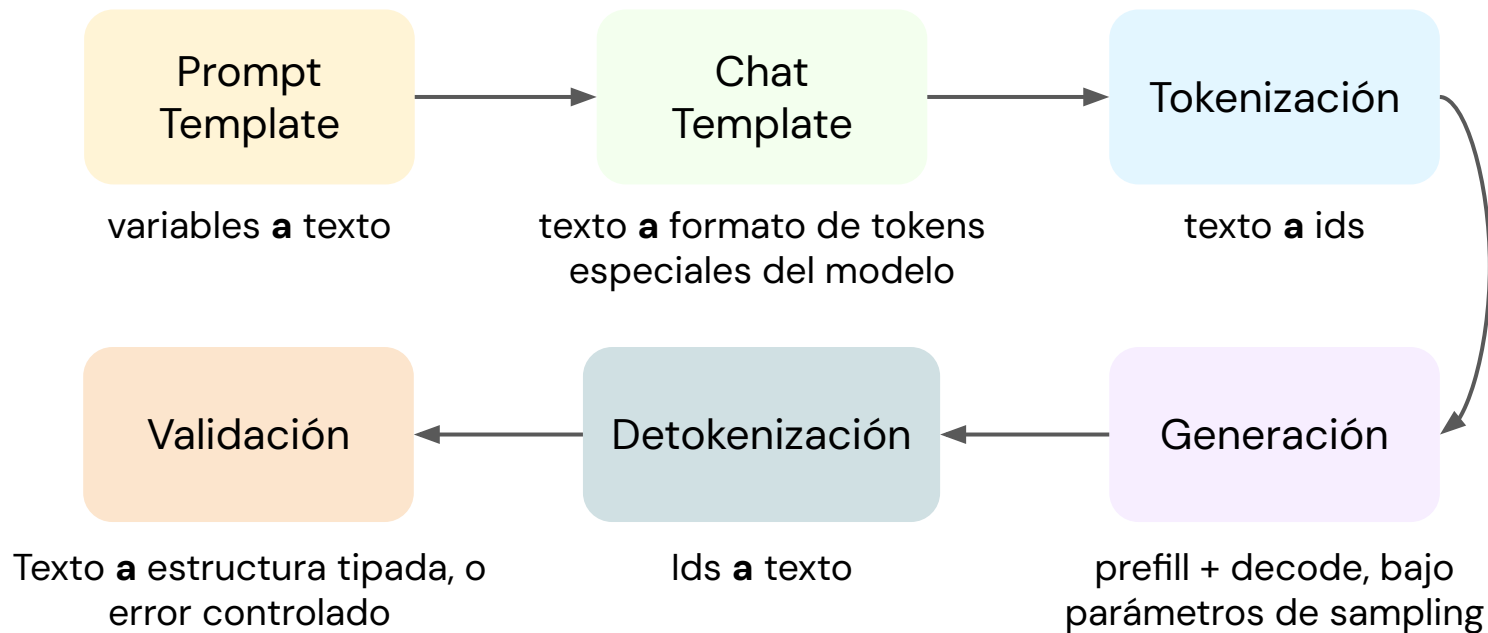
Anatomía de Inference Pipeline

Todo lo que ocurre entre el input del usuario y el output de nuestra aplicación:



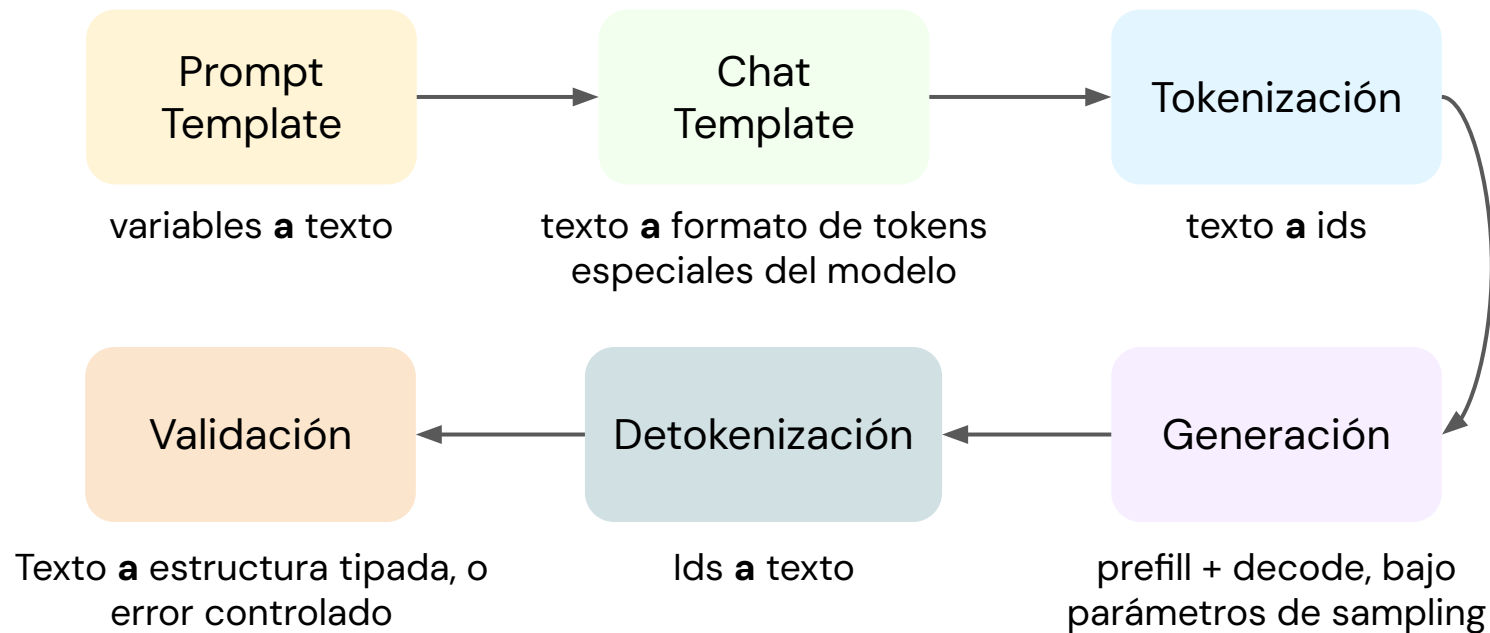
Anatomía de Inference Pipeline

Todo lo que ocurre entre el input del usuario y el output de nuestra aplicación:



Anatomía de Inference Pipeline

Todo lo que ocurre entre el input del usuario y el output de nuestra aplicación:



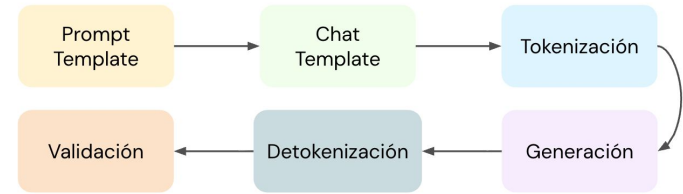
Cada uno de estos seis pasos es un artefacto versionable y un punto de falla independiente.

Ocho etapas de generate()

```
respuesta = modelo.generate(prompt)
```

El diagrama de seis cajas que acabamos de ver abre en ocho etapas:

1. **Prompt Template:** variables a texto (caja 1)
2. **Chat Template:** texto a formato del modelo (caja 2)
3. **Tokenización:** texto a ids (caja 3)
4. **Prefill:** ids a distribución
5. **Sampling:** distribución a un token
6. **Decode (bucle):** repetir 4 – 5 hasta EOS
7. **Detokenización:** ids a texto (caja 5)
8. **Validación**



Parametros de Sampling

El modelo entrega una distribución de probabilidad sobre todo el vocabulario. Los parámetros del modelo, permiten controlar el tipo de salida generada, especialmente su nivel de aleatoriedad.

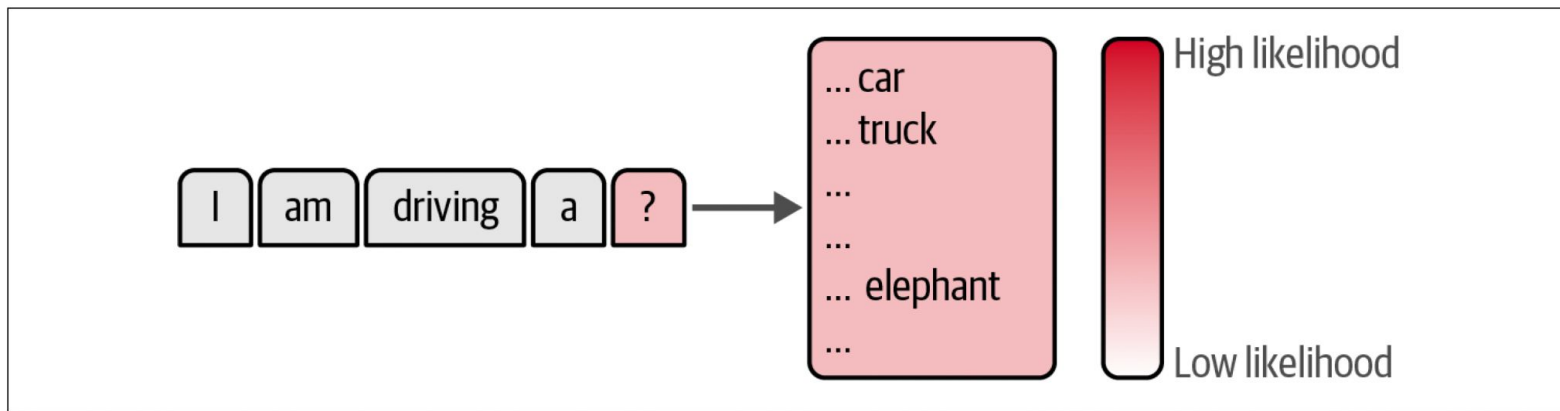


Figure 6-3. The model chooses the next token to generate based on their likelihood scores.

Referencia: Hands-On Large Language Models

Parametros de Sampling

Temperature: controla la aleatoriedad o creatividad del texto generado por un LLM.

- **Temperature = 0:** el modelo selecciona siempre el token más probable, por lo que tiende a generar la misma respuesta ante el mismo prompt.
- **Temperature alta:** permite seleccionar tokens menos probables, generando respuestas más variadas y creativas.

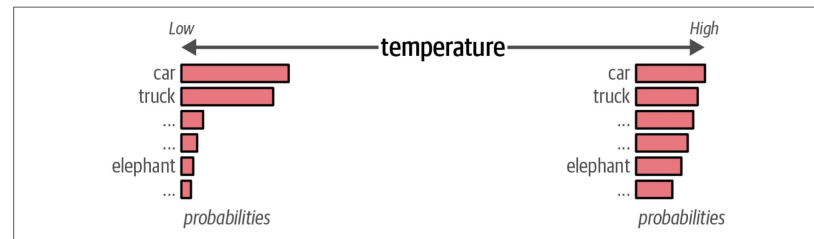


Figure 6-4. A higher *temperature* increases the likelihood that less probable tokens are generated and vice versa.

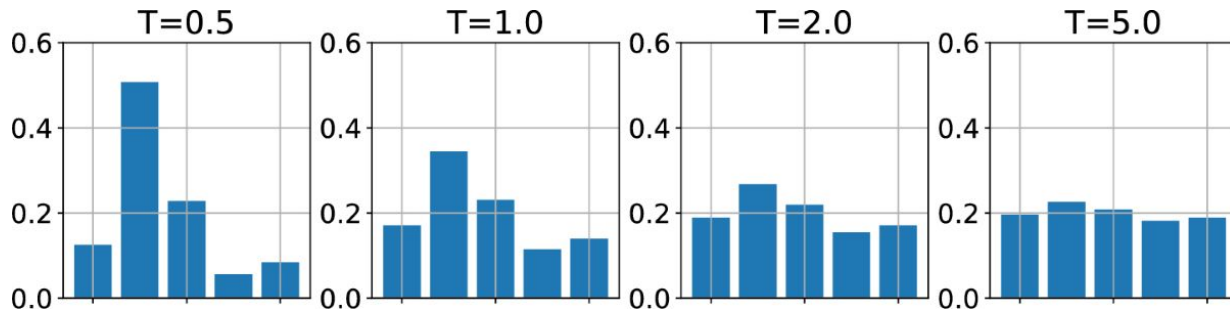
Referencia: Hands-On Large Language Models

Softmax

$$\frac{e^{x_i}}{\sum e^{x_j}}$$

Softmax con Temperatura

$$\frac{e^{\frac{x_i}{T}}}{\sum e^{\frac{x_j}{T}}}$$



Parametros de Sampling

Top_p (nucleus sampling): controla el conjunto de tokens que el modelo puede considerar al generar una respuesta.

El modelo ordena los tokens según su probabilidad y selecciona los que necesita hasta alcanzar la probabilidad acumulada definida por top_p.

- **top_p = 0.1:** considera únicamente los tokens más probables hasta alcanzar el 10% de probabilidad acumulada.
- **top_p = 1:** considera todos los tokens posibles.

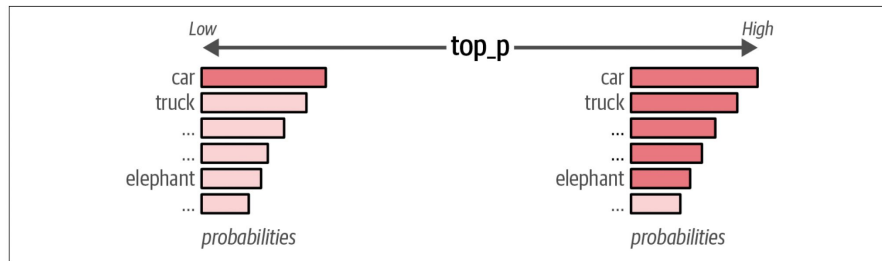
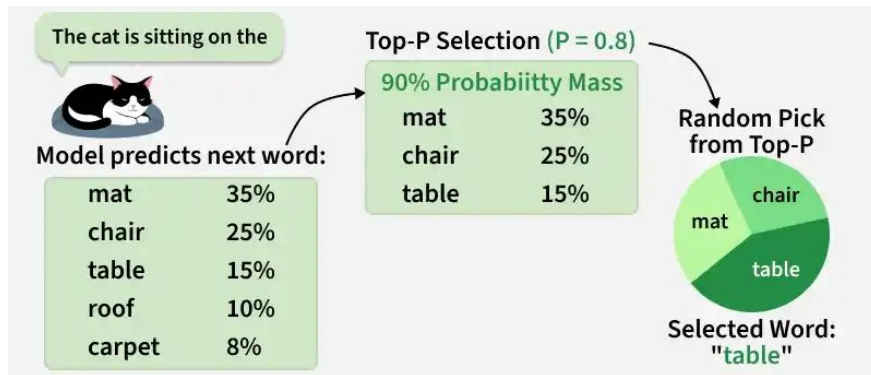


Figure 6-5. A higher top_p increases the number of tokens that can be selected to generate and vice versa.

Referencia: Hands-On Large Language Models



El orden importa

Los mismos parámetros, aplicados en distinto orden, dan distinto resultado:

`top_k(20) → top_p(0.9) ≠ top_p(0.9) → top_k(20)`

`temperatura(1.5) → top_p(0.9) ≠ top_p(0.9) → temperatura(1.5)`

- Hugging Face aplica temperatura antes de los filtros, otros runtimes no.
- Copiar `temperature=0.8`, `top_p=0.9` de un sistema a otro no garantiza el mismo comportamiento.
- No alcanza con registrar los parámetros: hay que registrar quién los interpreta.



Ejercicio

Parámetros de Sampling

[LINK](#)

El artefacto de una aplicación LLM

En MLOps clásico versionamos: código + datos + pesos.

En LLMOps, una configuración de inferencia reproducible requiere:

Un **artefacto** es cualquier elemento que conservamos y versionamos para poder reproducir exactamente un experimento, una evaluación o un despliegue de un LLM.

- **Identificador del modelo** (commit hash completo, del snapshot en disco)
- **Tokenizer**
- **Chat template**
- **Prompt template**
- **Parámetros de sampling completos**
- **Versión del runtime** (vLLM / Ollama / transformers) **y del hardware**
- **Configuración de recuperación** (índice, modelo de embeddings, top_k, chunking)
- **Dataset de evaluación y sus resultados**
- **Semilla (seed)** no elimina el no determinismo en GPU, pero sin ella ni siquiera podemos intentar comparar
- **Defaults efectivos de generation_config.json** (lo que el proveedor decidió por nosotros)

Estos campos son lo que vamos a registrar en MLflow

Reproducibilidad

Reproducibilidad

"Reproducible" tiene tres significados:

Término	Definición	Dónde se usa
Repetibilidad	Mismo equipo, mismo setup, mismo resultado	Laboratorio
Reproducibilidad	Otro equipo, mismo artefacto, mismo resultado	MLOps Clásico
Replicabilidad	Otro equipo, otro artefacto, misma conclusión	Ciencia



Reproducibilidad

El entrenamiento no es nuestro

Reproducimos la configuración de inferencia sobre un artefacto de terceros. Eso mueve tres cosas de lugar:

1. **No podemos reproducir el modelo:** No tenemos los datos, ni el cluster, ni la semilla del entrenamiento. Tenemos un puntero a un archivo. Lo máximo que podemos hacer es fijar ese puntero.
2. **No podemos verificar por igualdad:** El sistema no es determinista y con $\text{temperature} > 0$ lo rompemos por diseño.
3. **La dependencia crítica puede desaparecer:** El repositorio puede borrarse, cambiar de licencia, etc.

En MLOps clásico, con datos y código el modelo se reconstruye, en LLMOps no.

Reproducibilidad

Definición Operativa

Un entorno es reproducible si, a partir de artefactos versionados y sin intervención manual, podemos reconstruir un sistema cuya distribución de salidas es estadísticamente indistinguible de la original sobre nuestro dataset de evaluación.

- sin intervención manual: Documentado, no reproducible
- distribución: Comparamos tasas, medianas y percentiles, no strings
- sobre tu dataset: Sin dataset no hay afirmación

Escenario que queremos evitar

Situación: El equipo desarrolla en Colab. requirements.txt tiene **transformers** y **torch**, sin versiones. El modelo se carga con **from_pretrained("org/modelo")**. Se despliega tres semanas después en una VM con GPU.

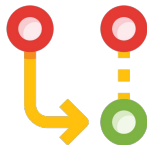


Resultado: El modelo carga, responde, y las respuestas son sistemáticamente peores. No hay error, no hay excepción, no hay alerta.



Causa real:

- Se publicó una versión nueva de transformers que cambió el default de un parámetro de generación.
- El repositorio del modelo recibió un commit que corrigió el chat template



Dependencias

Sobre requirements.txt

1. No es un lock (un lock busca congelar el conjunto exacto de dependencias y versiones que forman el entorno reproducible)

transformers>=4.40 instala cosas distintas hoy y en dos meses. Incluso **'==4.40.0'** deja libres las transitivas: tokenizers, safetensors, numpy.

2. No es reproducible por plataforma

Torch para Linux/CUDA 12 y para macOS/ARM son artefactos distintos. Un archivo plano no expresa eso.

3. No verifica integridad

Sin hashes, no sabemos si el paquete que bajamos es el que se publicó.

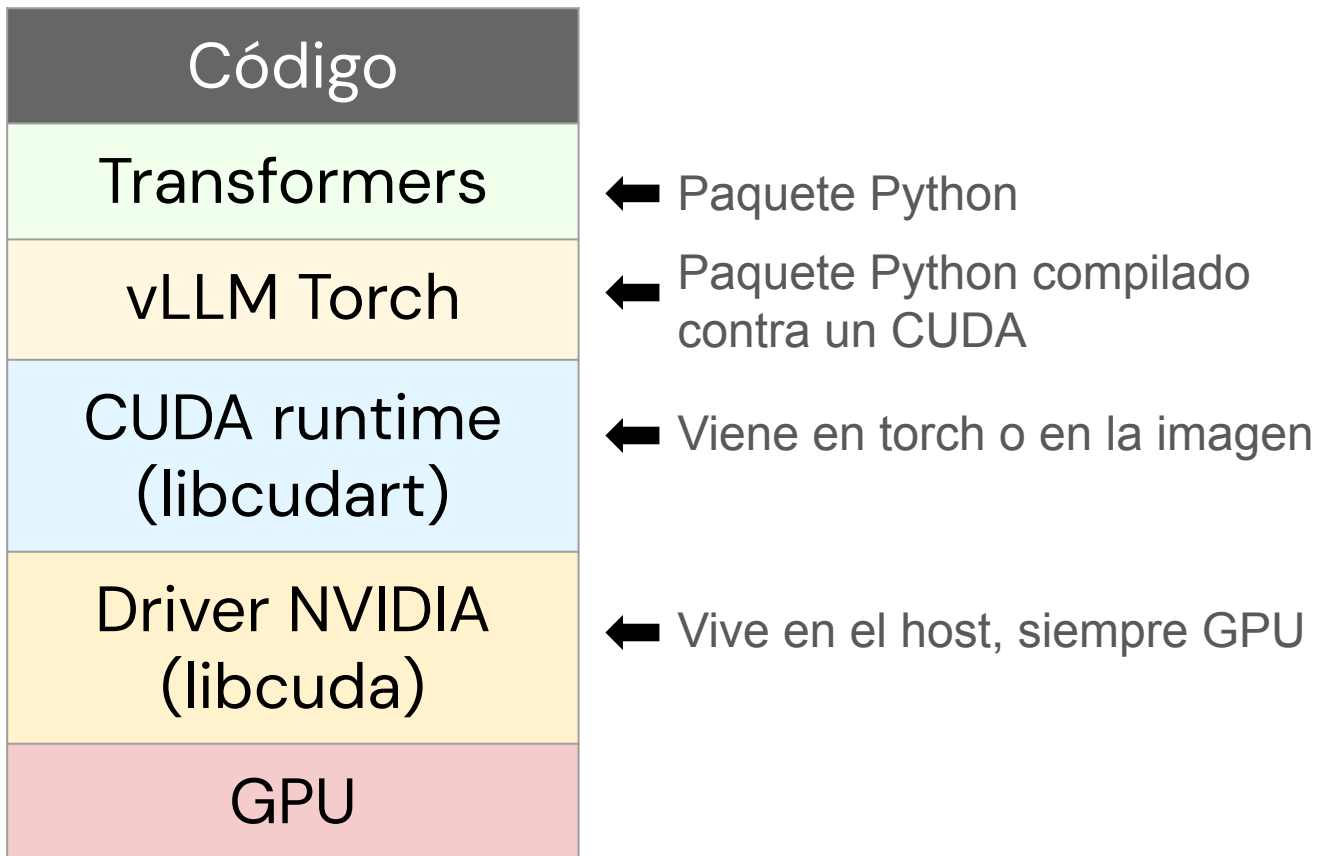
4. No expresa el stack nativo

CUDA, driver NVIDIA y cuDNN no son paquetes de Python. Son la mitad del problema y requirements.txt no los menciona.

Los resuelve
un gestor con
lock file

Lo resuelve un
contenedor

Stack Nativo





El manifiesto
es la receta

Manifiesto vs Lock

	Manifiesto	Lock
Qué es	Lo que pedimos	Lo que obtuvimos
Archivo	pyproject.toml	uv.lock / poetry.lock
Contenido	Restricciones amplias	Grafo completo resuelto, versiones exactas más hashes
Quién lo escribe	Un humano	La herramienta
Expresa	Intención	Un hecho
Se edita	Sí	Nunca
Se commitea	Sí	Siempre



El lock es la lista
de compras con
la marca y el
número de lote

No usar pip install suelto en un entorno que después vamos a desplegar: introduce paquetes que el lock no conoce y que nadie va a poder reconstruir.

Hash

Un hash es una huella digital matemática del contenido de un archivo. Si el archivo cambia, aunque sea mínimamente, su hash cambia.

Protege contra:

Índice: si alguien reemplaza un artefacto en el mirror interno

Transporte: corrupción, un proxy que cachea mal

Ambigüedad: dos ruedas con la misma versión y distinto contenido

No protege contra:

Vulnerabilidades: una versión fijada con hash es una versión que se queda vieja. El lock congela también los bugs conocidos.

Colab es Nivel 1 en dependencias

Los notebooks de esta clase corren `pip install transformers>=4.45 ...`. Eso es exactamente lo que la slide anterior dice que `no hay que hacer`.

Lo hacemos igual en los ejercicios de hoy porque:

- Colab impone su propio torch, compilado contra el CUDA de la VM. `Un lock nuestro entraría en conflicto con el runtime`, no lo arreglaría.
- El objetivo del notebook es didáctico, no desplegable.

Qué si “pinneamos” en el notebook:

- `La revisión del modelo (commit SHA)`, eso no depende de Colab
- `El HF_HOME`, dónde caen los pesos
- `Los parámetros de sampling` explícitos

Descarga y manejo de modelos

El Hub es Git

Extensión de Git que permite guardar archivos grandes sin meterlos directamente dentro del historial normal de Git



- Un repositorio de modelo es un repositorio git con LFS
- De ahí se derivan cuatro propiedades:
 - Tiene commits, cada uno con su SHA. **'main'** es una rama, y se mueve.
 - Tiene tags y ramas, los dos son punteros mutables.
 - Los archivos grandes van por LFS: se descargan y cachean aparte.
 - Cualquiera con permiso de escritura puede reescribir historia.

El Hub es Git

La idea es apuntar a un commit. Sin esto tendremos una dependencia flotante sobre un repositorio ajeno:

```
from_pretrained("org/modelo")
```

```
from_pretrained("org/modelo", revision="989aa79...")
```

El SHA corto funciona, pero se registra el completo (40 caracteres): el SHA que se registra es el que quedó en disco, no el que responde la API.

Esto aplica a las dos llamadas, no a una:

```
AutoTokenizer.from_pretrained("org/modelo", revision=SHA)
```

```
AutoModelForCausalLM.from_pretrained("org/modelo", revision=SHA)
```

Anatomía de un repositorio de modelo

config.json	Arquitectura, dimensiones
model-0000X-of-0000Y.safetensors	Los pesos, fragmentados
model.safetensors.index.json	Contiene el mapa para encontrar los pesos, no contiene los pesos del modelo.
tokenizer.json	El tokenizer serializado
tokenizer_config.json	Acá está el chat template
special_tokens_map.json	BOS, EOS, PAD, UNK
generation_config.json	Defaults de sampling del proveedor
README.md	Model card

Anatomía de un repositorio de modelo

tokenizer_config.json

- Si el proveedor corrige el chat template en un commit nuevo y seguimos en **main**, el formato del prompt cambia sin que toquemos nada.
- El modelo sigue respondiendo, pero mal.

generation_config.json

- Define temperature, top_p, eos_token_id por defecto. Si no los pasamos explícitos, el sampling lo decide en un archivo de un tercero.
- **eos_token_id** puede ser una lista.

safetensors vs pickle

Dos formas de guardar los pesos de un modelo de PyTorch:

	.pt / .bin	.safetensors
¿Qué guarda?	Objetos/tensores de PyTorch	Solo tensores
Seguridad	Puede ejecutar código con Pickle	No ejecuta código
Carga	Puede requerir más RAM	Generalmente más eficiente
Memory mapping	No es su enfoque	Sí
Uso actual	Menos recomendado	Preferido para LLMs
Idea clave	Flexible pero riesgoso	Seguro y eficiente

.pt/.bin: más flexible, pero Pickle puede ser peligroso.

.safetensors: solo datos de tensores, más seguro y eficiente.

Cache de Hugging Face

¿Dónde se guardan los modelos? Cuando usamos:

```
from transformers import AutoModel  
  
model = AutoModel.from_pretrained("...")
```

Hugging Face descarga y guarda los archivos en una caché para no volver a descargarlos.

¿Y si usamos Google Drive? Configurar la caché antes de importar Hugging Face para que los archivos se descarguen directamente en Google Drive.

```
import os  
  
os.environ["HF_HOME"] = "/content/hf_cache"  
  
from transformers import AutoModel
```

Hugging Face Transformers

Las tres formas de cargar un modelo:

1. `pipeline()`: una línea

```
pipe = pipeline("text-generation", model="org/modelo", revision=SHA)
```

Rápida para probar, esconde tokenización, chat template y sampling. No la usamos cuando lo que queremos es controlar el artefacto.

2. **Auto**: lo que usamos en el curso

```
tok = AutoTokenizer.from_pretrained("org/modelo", revision=SHA)
```

```
model = AutoModelForCausalLM.from_pretrained("org/modelo", revision=SHA,  
                                              dtype=torch.float16, device_map="auto",)
```

Los seis pasos del pipeline quedan visibles y versionables

3. Clase concreta (Ej: `LlamaForCausalLM`)

Solo si necesitamos algo que la Auto no expone

Transformers: Chat Template

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("mistralai/Mistral-7B-Instruct-v0.1")
chat = [
    {"role": "user", "content": "Hello, how are you?"},
    {"role": "assistant", "content": "I'm doing great. How can I help you today?"},
    {"role": "user", "content": "I'd like to show off how chat templating works!"},
]

tokenizer.apply_chat_template(chat, tokenize=False)
```

Cambia con un commit. Si no lo fijamos y no lo registramos, el formato del prompt cambia sin que toquemos una línea de código nuestro.

Ejercicio

Anatomía de un pipeline de inferencia

[LINK](#)

Model Registries y Versionado

El notebook produjo un número la semana pasada...

¿Con qué configuración, exactamente?

¿Podemos volver a esa configuración ahora mismo, sin adivinar?

Registro en LLMOps

- En MLOps clásico registramos un modelo entrenado.
- En LLMOps registramos una configuración:
 - Los pesos son una referencia dentro de esa configuración, no el objeto registrado.

Dos versiones del sistema pueden apuntar al mismo modelo y diferir solo en el prompt.

Esas dos versiones son tan distintas entre sí como dos modelos distintos, y hay que poder distinguirlas, compararlas y revertirlas igual.

Registro en LLMOps

Campo del artefacto	Cómo se registra
1. Modelo y revisión	Parámetro: repo_id y commit SHA completo
2. Tokenizer	Parámetro: repositorio y revisión
3. Chat template	Artefacto: el string
4. Prompt template	Prompt registry, con versión propia
5. Sampling	Parámetros: temperature, top_p, max_tokens, seed y los defaults efectivos que vinieron de generation_config.json
6. Runtime y hardware	Artefacto: entorno.json
7. Recuperación	Parámetros: índice, embeddings, top_k, chunking
8. Evaluación	Métricas y el dataset como artefacto versionado